

1 Statements

More interesting language — a language of simple statements:

$$\begin{aligned} \mathcal{S} = & \text{skip} \\ & \mathcal{X} := \mathcal{E} \\ & \text{read}(\mathcal{X}) \\ & \text{write}(\mathcal{E}) \\ & \mathcal{S}; \mathcal{S} \end{aligned}$$

Here $\mathcal{E}, \mathcal{X}$ stand for the sets of expressions and variables, as in the previous lecture. Informally, the language allows to write a straight-line programs which transform an input stream of integers into output stream of integers.

Again, we define the semantics for this language

$$\llbracket \bullet \rrbracket_{\mathcal{S}} : \mathcal{S} \mapsto \mathbb{Z}^* \rightarrow \mathbb{Z}^*$$

with the semantic domain of partial functions from integer streams to integer streams. Again, we will use *big-step operational semantics*: we define a ternary relation “ $\Rightarrow_{\mathcal{S}}$ ”

$$\Rightarrow_{\mathcal{S}} \subseteq \mathcal{C} \times \mathcal{S} \times \mathcal{C}$$

where $\mathcal{C}$ — a set of possible *configurations* during a program execution. We will write $c_1 \xRightarrow[\mathcal{S}]{} c_2$ instead of $(c_1, \mathcal{S}, c_2) \in \Rightarrow_{\mathcal{S}}$ and informally interpret the former as “the execution of a statement $\mathcal{S}$ in a configuration c_1 completes with the configuration c_2 ”.

The set of all configuration is defined as

$$\begin{aligned} \mathcal{C} &= \Sigma \times \mathcal{W} \\ \mathcal{W} &= \mathbb{Z}^* \times \mathbb{Z}^* \end{aligned}$$

where $\mathcal{W}$ — a set of *worlds*, each of which encapsulates some input-output stream pair. For simplicity, we define the following operations for worlds:

$$\begin{aligned} \text{read } \langle xi, o \rangle &= \langle x, \langle i, o \rangle \rangle \\ \text{write } x \langle i, o \rangle &= \langle i, ox \rangle \\ \text{out } \langle i, o \rangle &= o \end{aligned}$$

The relation “ $\Rightarrow_{\mathcal{S}}$ ” is defined by the following deductive system (see Fig. ??). The first three rules are *axioms* as they do not have any premises. Note, according to these rules sometimes a program cannot do a step in a given configuration: a value of an expression can be undefined in a given state in rules ASSIGN and WRITE, and there can be no input value in rule READ. This style of a semantics description is called big-step operational semantics, since the results of a computation are immediately observable at the right hand side of “ $\Rightarrow$ ” and, thus, the computation is performed in a single “big”

$$\begin{array}{c}
c \xRightarrow[\mathcal{I}]{\text{skip}} c \quad [\text{SKIP}] \\
\langle \sigma, \omega \rangle \xRightarrow[\mathcal{I}]{x := e} \langle \sigma[x \leftarrow \llbracket e \rrbracket_{\mathcal{E}} \sigma], \omega \rangle \quad [\text{ASSIGN}] \\
\frac{\langle z, \omega' \rangle = \text{read } \omega}{\langle \sigma, \omega \rangle \xRightarrow[\mathcal{I}]{\text{read}(x)} \langle \sigma[x \leftarrow z], \omega' \rangle} \quad [\text{READ}] \\
\langle \sigma, \omega \rangle \xRightarrow[\mathcal{I}]{\text{write}(e)} \langle \sigma, \text{write}(\llbracket e \rrbracket_{\mathcal{E}} \sigma) \omega \rangle \quad [\text{WRITE}] \\
\frac{c_1 \xRightarrow[\mathcal{I}]{S_1} c' \quad c' \xRightarrow[\mathcal{I}]{S_2} c_2}{c_1 \xRightarrow[\mathcal{I}]{S_1; S_2} c_2} \quad [\text{SEQ}]
\end{array}$$

Figure 1: Big-step operational semantics for statements

step. And, again, this style of a semantic description can be used to easily implement a reference interpreter.

With the relation “ $\Rightarrow_{\mathcal{I}}$ ” defined we can abbreviate the “surface” semantics for the language of statements:

$$\frac{\langle \Lambda, \langle i, \epsilon \rangle \rangle \xRightarrow[\mathcal{I}]{S} \langle \sigma, \omega \rangle}{\llbracket S \rrbracket_{\mathcal{I}} i = \text{out } \omega}$$

2 Stack Machine

Stack machine is a simple abstract computational device, which can be used as a convenient model to constructively describe the compilation process.

In short, stack machine operates on the same configurations, as the language of statements, plus a stack of integers. The computation, performed by the stack machine, is controlled by a program, which is described as follows:

$$\begin{array}{lcl}
\mathcal{I} & = & \text{BINOP } \otimes \\
& & \text{CONST } \mathbb{N} \\
& & \text{READ} \\
& & \text{WRITE} \\
& & \text{LD } \mathcal{X} \\
& & \text{ST } \mathcal{X} \\
\mathcal{P} & = & \epsilon \\
& & \mathcal{I} \mathcal{P}
\end{array}$$

Here the syntax category $\mathcal{I}$ stands for *instructions*, $\mathcal{P}$ — for *programs*; thus, a program is a finite string of instructions.

$$\begin{array}{c}
\frac{c \xRightarrow{\epsilon} \mathcal{SM} c}{\text{[STOP}_{SM}\text{]}} \\
\\
\frac{\langle (x \oplus y)s, c \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle yxs, c \rangle \xRightarrow{[\text{BINOP } \otimes] p} \mathcal{SM} c'} \quad [\text{BINOP}_{SM}] \\
\\
\frac{\langle zs, c \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle s, c \rangle \xRightarrow{[\text{CONST } z] p} \mathcal{SM} c'} \quad [\text{CONST}_{SM}] \\
\\
\frac{\langle z, \omega' \rangle = \mathbf{read } \omega, \langle zs, \langle \sigma, \omega' \rangle \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle s, \langle \sigma, \omega \rangle \rangle \xRightarrow{\text{READ } p} \mathcal{SM} c'} \quad [\text{READ}_{SM}] \\
\\
\frac{\langle s, \langle \sigma, \mathbf{write } z\omega \rangle \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle zs, \langle \sigma, \omega \rangle \rangle \xRightarrow{\text{WRITE } p} \mathcal{SM} c'} \quad [\text{WRITE}_{SM}] \\
\\
\frac{\langle (\sigma x)s, \langle \sigma, \omega \rangle \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle s, \langle \sigma, \omega \rangle \rangle \xRightarrow{[\text{LD } x] p} \mathcal{SM} c'} \quad [\text{LD}_{SM}] \\
\\
\frac{\langle s, \langle \sigma[x \leftarrow z], \omega \rangle \rangle \xRightarrow{p} \mathcal{SM} c'}{\langle zs, \langle \sigma, \omega \rangle \rangle \xRightarrow{[\text{ST } x] p} \mathcal{SM} c'} \quad [\text{ST}_{SM}]
\end{array}$$

Figure 2: Big-step operational semantics for stack machine

The semantics of stack machine program can be described, again, in the form of big-step operational semantics. This time the set of stack machine configurations is

$$\mathcal{C}_{SM} = \mathbb{Z}^* \times \mathcal{C}$$

where the first component is a stack, and the second — a configuration as in the semantics of statement language. The rules are shown on Fig. ??; note, now we have one axiom and six inference rules (one per instruction).

As for the statement, with the aid of the relation “ $\Rightarrow_{\mathcal{SM}}$ ” we can define the surface semantics of stack machine:

$$\frac{\langle \epsilon, \langle \Lambda, \langle i, \epsilon \rangle \rangle \rangle \xRightarrow{p} \mathcal{SM} \langle s, \langle \sigma, \omega \rangle \rangle}{\llbracket p \rrbracket_{\mathcal{SM}} i = \mathbf{out } \omega}$$

3 A Compiler for the Stack Machine

A compiler of the statement language into the stack machine is a total mapping

$$\llbracket \bullet \rrbracket^{comp} : \mathcal{S} \mapsto \mathcal{P}$$

We can describe the compiler in the form of denotational semantics for the source language. In fact, we can treat the compiler as a *static* semantics, which maps each program into its stack machine equivalent.

As the source language consists of two syntactic categories (expressions and statements), the compiler has to be “bootstrapped” from the compiler for expressions $\llbracket \bullet \rrbracket_{\mathcal{E}}^{comp}$:

$$\begin{aligned} \llbracket x \rrbracket_{\mathcal{E}}^{comp} &= [\text{LD } x] \\ \llbracket n \rrbracket_{\mathcal{E}}^{comp} &= [\text{CONST } n] \\ \llbracket A \otimes B \rrbracket_{\mathcal{E}}^{comp} &= \llbracket A \rrbracket_{\mathcal{E}}^{comp} \llbracket B \rrbracket_{\mathcal{E}}^{comp} [\text{BINOP } \otimes] \end{aligned}$$

And now the main dish:

$$\begin{aligned} \llbracket x := e \rrbracket^{comp} &= \llbracket e \rrbracket_{\mathcal{E}}^{comp} [\text{ST } x] \\ \llbracket \text{read } (x) \rrbracket^{comp} &= [\text{READ}] [\text{ST } x] \\ \llbracket \text{write } (e) \rrbracket^{comp} &= \llbracket e \rrbracket_{\mathcal{E}}^{comp} [\text{WRITE}] \\ \llbracket S_1 ; S_2 \rrbracket^{comp} &= \llbracket S_1 \rrbracket^{comp} \llbracket S_2 \rrbracket^{comp} \end{aligned}$$